

RAPPORT DETAILLE DU PROJET

1. RESUME DU PROJET

Ce projet est une application web full-stack pour la gestion d'accès utilisateur dans un contexte professionnel interne, probablement lié à Yazaki. L'application permet à des utilisateurs métier de demander un compte, puis impose une validation manuelle par un administrateur avant d'autoriser la connexion.

L'état actuel du projet montre une base solide d'authentification et de gestion des rôles, mais les espaces métier finaux ne sont pas encore développés. Pour le moment, la partie la plus aboutie est le flux d'inscription, de connexion, de validation des comptes et de redirection selon le rôle.

2. OBJECTIF FONCTIONNEL

L'objectif principal semble être de fournir une application interne où :

- un utilisateur peut demander un compte avec un rôle précis ;
- seuls les emails professionnels '@yazaki-europe.com' sont acceptés ;
- un administrateur peut approuver ou rejeter les demandes ;
- seuls les comptes approuvés peuvent se connecter ;
- chaque rôle est redirigé vers son propre espace.

Les rôles actuellement pris en charge sont :

- 'admin'
- 'directeur'
- 'gestionnaire'

Dans l'interface, 'gestionnaire' correspond à 'gestionnaire de stock'.

3. ETAT ACTUEL DU PRODUIT

Le projet est fonctionnel pour :

- l'inscription d'un directeur ou gestionnaire ;
- la validation stricte de l'identifiant professionnel Yazaki ;
- la création d'un compte en attente d'approbation ;
- la connexion avec JWT ;
- la restauration de session côté frontend via '/api/users/me' ;
- la protection des routes selon le rôle ;
- la consultation des demandes en attente par l'administrateur ;
- l'approbation ou le rejet d'un compte en attente ;
- la redirection vers un espace dédié selon le rôle.

Le projet n'est pas encore fonctionnel pour :

- un vrai dashboard directeur ;
- un vrai dashboard gestionnaire de stock ;
- des modules metier comme stocks, inventaire, KPI, rapports, commandes, etc.

Actuellement, les pages 'directeur' et 'gestionnaire-stock' sont des placeholders.

4. ARCHITECTURE TECHNIQUE

Le projet est separe en deux dossiers principaux :

- 'Backend/' : API Node.js / Express / MongoDB
- 'frontend/' : application React avec Vite

Backend

Pile utilisee :

- Node.js
- Express
- MongoDB avec Mongoose
- JWT pour l'authentification
- bcryptjs pour le hash des mots de passe
- dotenv pour la configuration
- cors pour l'acces frontend

Frontend

Pile utilisee :

- React 19
- React Router DOM 7
- Vite
- lucide-react pour les icones
- CSS custom, sans framework CSS externe

5. ARCHITECTURE FONCTIONNELLE

5.1 Flux d'inscription

Un utilisateur remplit le formulaire d'inscription avec :

- nom
- prenom
- identifiant professionnel
- role
- mot de passe
- confirmation du mot de passe

Regles appliquees :

- seuls les roles 'directeur' et 'gestionnaire' sont autorises a l'inscription ;
- l'utilisateur doit saisir uniquement la partie avant '@' ;
- le domaine '@yazaki-europe.com' est ajoute automatiquement ;
- le mot de passe doit contenir au moins 8 caracteres, une majuscule, une minuscule et un chiffre ;
- le compte cree est automatiquement en attente de validation ;
- un compte deja existant avec le meme email est refuse.

5.2 Flux de validation admin

L'administrateur voit la liste des utilisateurs :

- non approuves
- non rejetes
- ayant un role 'directeur' ou 'gestionnaire'

Il peut :

- approuver un compte ;
- rejeter un compte.

Effets :

- 'approve' => 'isApproved = true', 'isRejected = false'
- 'reject' => 'isApproved = false', 'isRejected = true'

5.3 Flux de connexion

Lors de la connexion :

- l'identifiant est normalise ;
- l'utilisateur est recherche par email ;
- le mot de passe est compare avec le hash bcrypt ;
- si le compte est rejete, la connexion est refusee ;
- si le compte n'est pas encore approuve, la connexion est refusee ;
- si tout est valide, le backend renvoie un JWT valable 30 jours.

5.4 Flux de session frontend

Le frontend :

- stocke le token JWT dans 'localStorage' sous la cle 'pfe_auth_token' ;
- l'ajoute dans l'entete 'Authorization: Bearer ...' pour chaque requete ;
- appelle '/api/users/me' au chargement pour restaurer la session ;
- redirige l'utilisateur selon son role.

6. STRUCTURE DU BACKEND

Point d'entree

Fichier principal :

- 'Backend/server.js'

Responsabilites :

- charge les variables d'environnement ;
- configure CORS avec 'FRONTEND_URL' ;
- active 'express.json()' ;
- monte les routes utilisateur sous '/api/users' ;
- lance la connexion MongoDB ;
- demarre le serveur.

Base de donnees

Fichier :

- 'Backend/config/db.js'

Role :

- connexion a MongoDB via 'mongoose.connect(process.env.MONGO_URI)'.

Modele utilisateur

Fichier :

- 'Backend/models/User.js'

Schema utilisateur :

- 'username: String'
- 'email: String'
- 'password: String'
- 'role: "gestionnaire" | "directeur" | "admin"'
- 'isApproved: Boolean'
- 'isRejected: Boolean'
- 'createdAt', 'updatedAt'

Regles importantes :

- email unique ;
- username unique ;
- mot de passe hashe avant sauvegarde avec bcrypt ;

- 'isApproved' est automatique a 'true' pour 'admin', sinon faux par default.

Middleware d'authentification

Fichier :

- 'Backend/middleware/auth.js'

Contient :

- 'protect' : verifie le token JWT, charge l'utilisateur courant ;
- 'adminOnly' : limite l'accès aux administrateurs.

Routes d'authentification et de gestion utilisateur

Fichier :

- 'Backend/routes/auth.js'

Routes disponibles :

- 'POST /api/users/register'
- 'POST /api/users/login'
- 'GET /api/users/me'
- 'GET /api/users/admin-data'
- 'GET /api/users/pending'
- 'PATCH /api/users/pending/:userId/approve'
- 'PATCH /api/users/pending/:userId/reject'

Validation des identifiants Yazaki

Fichier :

- 'Backend/utills/yazakiEmail.js'

Ce module :

- verifie que l'identifiant ne contient pas d'espaces ;
- accepte soit la partie locale seule, soit l'email complet selon le contexte ;
- impose le domaine 'yazaki-europe.com' ;
- reconstruit l'email final ;
- fournit des messages d'erreur métier clairs.

Creation d'admin

Il existe deux scripts :

- 'Backend/scripts/createAdmin.js'

- 'Backend/createAdmin.js'

Le script le plus propre et le plus recent semble etre :

- 'Backend/scripts/createAdmin.js'

Il prend en charge :

- 'ADMIN_IDENTIFIER' ou 'ADMIN_EMAIL'
- 'ADMIN_PASSWORD'

Le fichier 'Backend/createAdmin.js' ressemble plutot a une ancienne version de test, avec :

- email fixe 'admin@yazaki-europe.com'
- mot de passe fixe 'admin123'
- texte avec problemes d'encodage

7. STRUCTURE DU FRONTEND

Point d'entree

Fichiers :

- 'frontend/src/main.jsx'
- 'frontend/src/App.jsx'

Le frontend :

- utilise 'BrowserRouter' ;
- encapsule l'application dans 'AuthProvider' ;
- declare les routes publiques et protegees.

Gestion de l'authentification

Fichiers :

- 'frontend/src/auth/AuthContext.jsx'
- 'frontend/src/auth/useAuth.js'
- 'frontend/src/auth/auth-context.js'
- 'frontend/src/api/authApi.js'

Responsabilites :

- restaurer la session au chargement ;
- exposer 'login', 'logout', 'user', 'authReady', 'isAuthenticated' ;
- centraliser les appels API ;
- stocker le token dans 'localStorage'.

Routes frontend

Routes publiques :

- '/login'
- '/register'

Routes admin :

- '/admin'
- '/admin/dashboard'
- '/admin/approvals'

Routes directeur :

- '/directeur'
- '/directeur/dashboard'

Routes gestionnaire :

- '/gestionnaire-stock'
- '/gestionnaire-stock/dashboard'

Autres :

- '/' redirige vers '/login'
- '*' affiche une page 404

Protection des routes

Fichiers :

- 'frontend/src/components/PublicOnlyRoute.jsx'
- 'frontend/src/components/ProtectedRoute.jsx'
- 'frontend/src/components/AdminProtectedRoute.jsx'

Comportement :

- un utilisateur non connecté est renvoyé vers '/login' ;
- un utilisateur connecté ne peut plus voir '/login' ou '/register' ;
- un utilisateur connecté mais avec le mauvais rôle est redirigé vers sa page d'accueil ;
- l'application attend la fin de la restauration de session avant de statuer.

Pages principales

Login

Fichier :

- 'frontend/src/pages/LoginPage.jsx'

Fonctions :

- saisie identifiant + mot de passe ;
- affichage des erreurs métier du backend ;
- redirection selon le rôle après connexion.

Register

Fichiers :

- 'frontend/src/pages/RegisterPage.jsx'
- 'frontend/src/components/RegisterForm.jsx'
- 'frontend/src/Utils/registerValidation.js'

Fonctions :

- formulaire détaillé d'inscription ;
- validation locale des noms, rôle, mot de passe et identifiant Yazaki ;
- envoi d'un message de succès puis redirection vers la page de connexion.

Admin approvals

Fichier :

- 'frontend/src/pages/AdminApprovalsPage.jsx'

Fonctions :

- chargement des utilisateurs en attente ;
- affichage du nom, email et rôle ;
- boutons d'acceptation et de rejet ;
- mise à jour locale de la liste après action.

Placeholders métier

Fichiers :

- 'frontend/src/pages/DirectorPlaceholderPage.jsx'
- 'frontend/src/pages/StockManagerPlaceholderPage.jsx'

Fonctions :

- afficher un espace d'accueil simple selon le rôle ;

- proposer un bouton de deconnexion ;
- servir de base a de futurs dashboards.

Style et experience utilisateur

Fichiers :

- 'frontend/src/index.css'
- 'frontend/src/App.css'

Le design actuel :

- est propre et moderne ;
- utilise une dominante rouge, probablement liee a Yazaki ;
- propose des cartes, gradients, formulaires responsives ;
- gere correctement l'affichage mobile ;
- fournit des etats de chargement et des messages d'erreur.

8. REGLES METIER IMPORTANTES

Voici les regles metier a transmettre absolument a une autre IA :

- seuls les comptes avec domaine '@yazaki-europe.com' sont autorises ;
- pour l'inscription, seuls 'directeur' et 'gestionnaire' sont autorises ;
- les comptes 'admin' ne s'inscrivent pas via le formulaire standard ;
- tout nouveau compte non admin doit etre approuve manuellement ;
- un compte rejete ne peut pas se connecter ;
- un compte non approuve ne peut pas se connecter ;
- le role determine la page de destination apres connexion ;
- 'gestionnaire' est le role backend, mais l'interface l'affiche comme 'gestionnaire de stock'.

9. VARIABLES D'ENVIRONNEMENT

Backend

Variables utilisees :

- 'PORT'
- 'FRONTEND_URL'
- 'MONGO_URI'
- 'JWT_SECRET'
- 'ADMIN_IDENTIFIER'
- 'ADMIN_EMAIL'
- 'ADMIN_PASSWORD'

Frontend

Variables utilisees :

- 'VITE_API_BASE_URL'

Un exemple existe dans :

- 'frontend/.exemple.env'

Valeur actuelle d'exemple :

- 'VITE_API_BASE_URL=http://localhost:5000'

10. SCRIPTS DISPONIBLES

Backend

Dans 'Backend/package.json' :

- 'npm run dev'
- 'npm start'
- 'npm run seed:admin'

Frontend

Dans 'frontend/package.json' :

- 'npm run dev'
- 'npm run build'
- 'npm run lint'
- 'npm run preview'

11. LIMITES ET POINTS D'ATTENTION

Fonctionnalites encore inachevees

- pas de module de gestion de stock reel ;
- pas de dashboard directeur reel ;
- pas de CRUD metier au-dela des utilisateurs ;
- pas de tableaux de bord, KPI, reporting, ou historique ;
- pas de gestion de profil utilisateur avancee.

Dette technique / incoherences observees

- 'frontend/src/pages/AdminPage.jsx' semble etre une ancienne page admin, differente du flux actuellement utilise ;
- 'Backend/createAdmin.js' semble etre une ancienne version moins propre du script de creation admin ;
- il y a quelques traces de problemes d'encodage dans certains messages ;

- le frontend envoie 'credentials: 'include'', mais l'authentification réelle repose sur un token JWT dans 'localStorage', pas sur des cookies ;
- pas de suite de tests automatisée détectée ;
- pas de gestion avancée des erreurs réseau ni de journalisation structurée ;
- la logique de validation des identifiants Yazaki est dupliquée en backend et frontend, ce qui est utile pour l'UX mais impose de maintenir les deux versions en parallèle.

Securite

Points positifs :

- hash bcrypt des mots de passe ;
- vérification JWT ;
- protection des routes admin ;
- restriction CORS à une origine définie ;
- validation métier stricte de l'identifiant professionnel.

Points à surveiller :

- stockage du JWT dans 'localStorage' ;
- absence visible de refresh token ;
- durée de validité du token de 30 jours ;
- script legacy avec mot de passe admin statique si jamais il est encore utilisé.

12. FICHIERS LES PLUS IMPORTANTS

Si une autre IA doit comprendre rapidement le projet, elle doit lire en priorité :

- 'Backend/server.js'
- 'Backend/routes/auth.js'
- 'Backend/models/User.js'
- 'Backend/middleware/auth.js'
- 'Backend/utls/yazakiEmail.js'
- 'Backend/scripts/createAdmin.js'
- 'frontend/src/App.jsx'
- 'frontend/src/api/authApi.js'
- 'frontend/src/auth/AuthContext.jsx'
- 'frontend/src/config/roles.js'
- 'frontend/src/components/RegisterForm.jsx'
- 'frontend/src/pages/LoginPage.jsx'
- 'frontend/src/pages/AdminApprovalsPage.jsx'
- 'frontend/src/utls/registerValidation.js'

13. RESUME ULTRA-COURT POUR UNE IA

Ce projet est une application web interne en React + Node/Express + MongoDB pour gérer l'inscription, la connexion et la validation de comptes utilisateurs professionnels Yazaki. Les rôles sont 'admin', 'directeur' et 'gestionnaire'. Les utilisateurs

'directeur' et 'gestionnaire' peuvent demander un compte, mais un 'admin' doit valider manuellement leur demande avant qu'ils puissent se connecter. L'authentification se fait par JWT. Le backend est déjà opérationnel pour auth + approbation, mais les dashboards métier sont encore des placeholders.

14. PROMPT CONSEILLE A ENVOYER A CHATGPT OU GEMINI

Tu vas m'aider à continuer un projet full-stack déjà existant. Voici le contexte complet :

Mon projet est une application web interne pour Yazaki, construite avec un frontend React/Vite et un backend Node.js/Express avec MongoDB/Mongoose. Le système gère l'inscription, la connexion et l'approbation manuelle des comptes utilisateurs.

Roles :

- admin
- directeur
- gestionnaire (affiche comme gestionnaire de stock dans le frontend)

Regles métier :

- seuls les emails professionnels '@yazaki-europe.com' sont autorisés ;
- seuls les rôles directeur et gestionnaire peuvent s'inscrire via le formulaire ;
- les comptes admin sont créés à part ;
- tout nouveau compte non admin doit être approuvé manuellement par un administrateur ;
- un compte non approuvé ou rejeté ne peut pas se connecter ;
- après connexion, chaque rôle est redirigé vers son propre espace.

Backend :

- Express
- Mongoose
- JWT
- bcryptjs
- routes principales : register, login, me, pending, approve, reject

Frontend :

- React
- React Router
- AuthContext pour restaurer la session
- token JWT stocké dans localStorage
- page admin pour valider ou rejeter les comptes
- pages directeur et gestionnaire encore en mode placeholder

Je veux que tu analyses cette architecture comme un développeur senior et que tu m'aides à [mettre ici ton besoin précis].

Quand tu proposes du code :

- respecte l'architecture existante ;
- conserve les noms de rôles actuels ;

- ne casse pas le flux d'approbation admin ;
- privilegie des changements progressifs et coherents avec le code deja present.

15. CONCLUSION

Le projet est une bonne base d'application interne securisee autour d'un workflow d'approbation utilisateur. Le coeur de l'application aujourd'hui n'est pas encore la gestion de stock elle-meme, mais plutot la gestion des acces et des roles. La prochaine etape logique serait de construire les vraies fonctionnalites metier dans les espaces 'directeur' et 'gestionnaire-stock'.